

Express.js: Day 1

Express fundamentals: application, routes, request/response, server. Long-term reference notes.

Day 1 goal

Understand the complete basic Express flow: import Express, create the app, define a route, receive a request, run the handler, send a response, listen on a port.

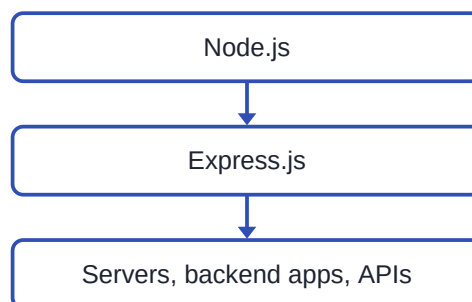
Course position: Node.js is complete. This is the first day of the Express.js section.

What you learned today

Topic	Core idea
Express.js	Node.js web framework for building servers and backend applications/APIs.
express vs app	<code>express</code> = imported package functionality; <code>app</code> = Express application instance.
Routes	HTTP method + path + handler.
req / res	Incoming request information vs outgoing response.
app.listen()	Starts listening for incoming connections on a port.
Request to response	Client request, route match, handler, response, back to client.

What is Express.js?

Express.js is a web framework for Node.js. It gives you a convenient way to build HTTP servers, backend applications and APIs without handling all the low-level HTTP details every time.



Mental model

Node.js gives the runtime and low-level capabilities. Express adds a structured, convenient layer for routes, requests, responses and middleware.

Importing Express

Express is distributed through npm. In a CommonJS Node.js project, load it like this:

```
const express = require("express");
```

This loads the Express package and stores what it exports in a variable named `express`. The name is only a convention; what matters is what `require("express")` returns.

Terminology

Express = the Node.js web framework.

`express` = the variable holding the functionality returned by importing the package.

express vs app

```
const express = require("express");
const app = express();
```

express	app
Imported Express functionality	Express application instance
Used to create the application	Used to configure the application



Remember

`app.get()`, `app.use()` and `app.listen()` all operate on the Express application stored in `app`.

Routes and app.get()

A route tells Express what to do when a particular HTTP method and path are requested.

```
app.get("/", (req, res) => {  
  res.send("Home");  
});
```

Basic structure: `app.get(path, handler);`

Route = HTTP method + path + handler

- GET + `"/"` + handler
- GET + `"/about"` + handler
- GET + `"/profile"` + handler

```
app.get("/", (req, res) => {  
  res.send("Home Page");  
});  
  
app.get("/about", (req, res) => {  
  res.send("About Page");  
});  
  
app.get("/contact", (req, res) => {  
  res.send("Contact Page");  
});
```

GET requests

GET is an HTTP method generally used when the client wants to retrieve information. When you visit a URL in a browser, the browser generally makes a GET request, for example `GET /about`.

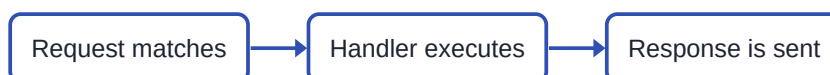
Express looks for a registered route whose HTTP method and path match the incoming request.

Route handler

```
app.get("/about", (req, res) => {  
  res.send("About Page");  
});
```

The function below is the route handler. Express calls it when the incoming request matches the route.

```
(req, res) => {  
  res.send("About Page");  
}
```



Request (req) and response (res)

Core idea

`req` = information about what the client sent to the server.

`res` = the object the server uses to send a response back to the client.

```
app.get("/about", (req, res) => {  
  console.log(req.method);  
  console.log(req.url);  
  res.send("About Page");  
});
```

Request information includes the HTTP method and URL. Later you will learn `req.params`, `req.query`, `req.body` and `req.headers`.

Memory aid: req = incoming, res = outgoing.

res.send()

`res.send()` sends a response back to the client.

```
app.get("/profile", (req, res) => {  
  res.send("This is my profile");  
});
```

The browser displays the response text. On Day 1 you tested this route successfully in the browser.

localhost and ports

`http://localhost:3000`

localhost	The local machine (this computer)
3000	The port where the application is listening

Port 3000 is common for development but not mandatory. Any available port works.

```
app.listen(4000); // another possible development port
```

app.listen()

`app.listen()` starts the server listening for incoming connections on the given port.

```
app.listen(3000);
```

A common form includes a callback, which runs after the server starts listening:

```
app.listen(3000, () => {  
  console.log("Server started");  
});
```

Don't confuse these

`app.get()` defines what to do for a matching GET request.

`app.listen()` starts listening for connections on a port.

Complete Express application

```
const express = require("express");
const app = express();

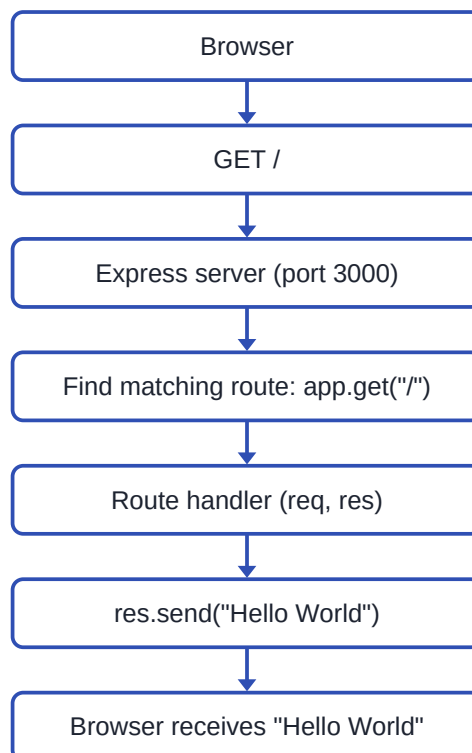
app.get("/", (req, res) => {
  res.send("Hello World");
});

app.listen(3000, () => {
  console.log("Server running on port 3000");
});
```

Line by line:

- 1 `require("express")` loads the Express package.
- 2 `express()` creates an Express application instance.
- 3 `app.get("/")` defines a GET route for the root path.
- 4 `(req, res) => { ... }` is the route handler.
- 5 `res.send("Hello World")` sends the response.
- 6 `app.listen(3000)` starts listening on port 3000.

Complete request to response flow



When you type `http://localhost:3000`, the browser requests the root path `/`. The server receives the GET request, Express finds `app.get("/")`, runs its handler, and the response goes back to the browser.

The story to remember

Import Express, create app, define route, listen on port. Then: client sends request, Express matches route, handler runs, response goes back to client.

What if the route doesn't exist?

If you define routes for `/` and `/about` and request `/products`, no GET route matches. You will typically see:

```
Cannot GET /products
```

Express received the request but found no registered GET route for that path. You tested this yourself in the browser on Day 1.

Important distinctions

Term	Meaning
<code>express</code>	Imported Express functionality
<code>app</code>	Express application instance
<code>app.get()</code>	Registers a GET route
<code>req</code>	Incoming request information
<code>res</code>	Response object used to send data back
<code>res.send()</code>	Sends a response
<code>app.listen()</code>	Starts listening for connections
<code>localhost</code>	This / local machine
<code>3000</code>	Port number used by the local server

Common beginner mistakes

Mistake	Reality
Confusing <code>express</code> and <code>app</code>	<code>express</code> is the imported functionality; <code>app</code> is the application instance you configure.
Thinking <code>app.get()</code> starts the server	<code>app.get()</code> defines a route. <code>app.listen()</code> starts listening.
Mixing up <code>req</code> and <code>res</code>	<code>req</code> is incoming request info; <code>res</code> sends the response.
Thinking every URL works automatically	A path needs a matching route (or other Express handling) to give the intended response.
Thinking port 3000 is mandatory	3000 is a common development choice, not a requirement.

Practice completed on Day 1

You created and tested routes yourself:

```
app.get("/", (req, res) => {
  res.send("Hello Bhavana");
});

app.get("/profile", (req, res) => {
  res.send("This is my profile");
});

app.listen(3000, () => {
  console.log("Server started at 3000 port");
});
```

You verified `/profile` in the browser and got the expected response. You also tested an undefined route and saw `Cannot GET /contact`.

Assessment result

Day 1 assessment passed. You correctly explained routes, route handlers, `req`, `res`, `app.listen()`, ports and the complete request-response flow.

Day 1 quick recap

- Express.js is a Node.js web framework.
- `require("express")` loads the Express package.
- `express()` creates an Express application instance.
- `app` stores that application instance.
- `app.get(path, handler)` registers a GET route.
- `req` represents incoming request information.
- `res` is used to send the response.
- `res.send()` sends a response to the client.
- `localhost` means the local machine.
- `app.listen(3000)` starts listening on port 3000.
- A request must match the HTTP method and path for the route handler to run.

Interview / self-check questions

- 1 What is Express.js and why do we use it with Node.js?
- 2 What is the difference between `express` and `app` ?
- 3 What happens when `const app = express()` executes?
- 4 What is a route?
- 5 What does `app.get()` do?
- 6 What is a route handler?
- 7 What are `req` and `res` ?
- 8 What does `res.send()` do?
- 9 What is `localhost` ?
- 10 What is a port?
- 11 What is the difference between `app.get()` and `app.listen()` ?
- 12 Explain the complete request, route, handler, response flow.

Express.js Day 1: complete

Foundation: Express application, routes, GET requests, handlers, request/response, localhost, ports and server startup. Next: Day 2 builds on this, one concept at a time.